

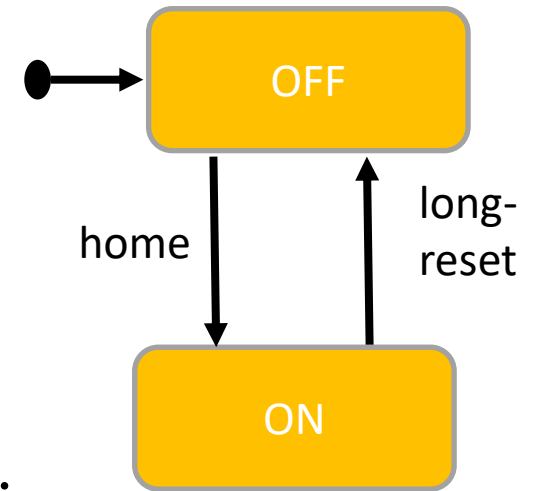
Software Testing

Course's Code: CSE 453

Test Design Techniques: State Transition Testing
(Chapter 4)

State Models: What is? and How to Draw?

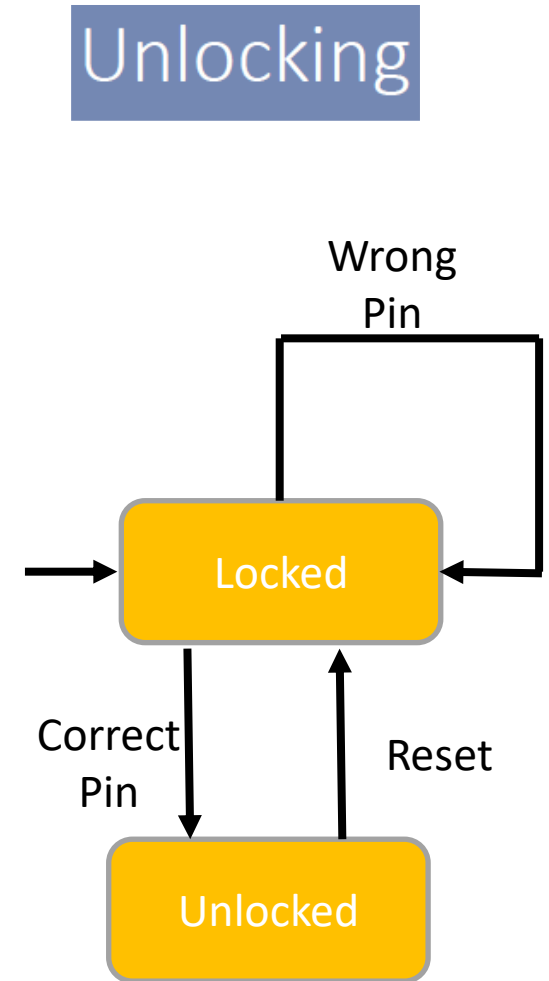
- A State Model represents behavior of a system
- Most systems around us have some notion of "state" inside them.
- For example, your phone can be switched off, in stand-by mode, or in use.
- Let's model that behavior using a state machine. Here we see a state in the orange box, labeled "OFF".
- If we switch it on using the "home" button, we enter a state labeled "ON".
- Once the phone is on, we can also switch it off. This leads to another transition, this time triggered pressing the "reset" button for a few seconds.
- With two states, we should decide which is the "initial" state. For the phone we will assume that the phone initially is switched off.
- This very simple model represents one aspect of the phone's behavior: Namely how it can be switched on and off using the home and reset buttons.



On versus Off

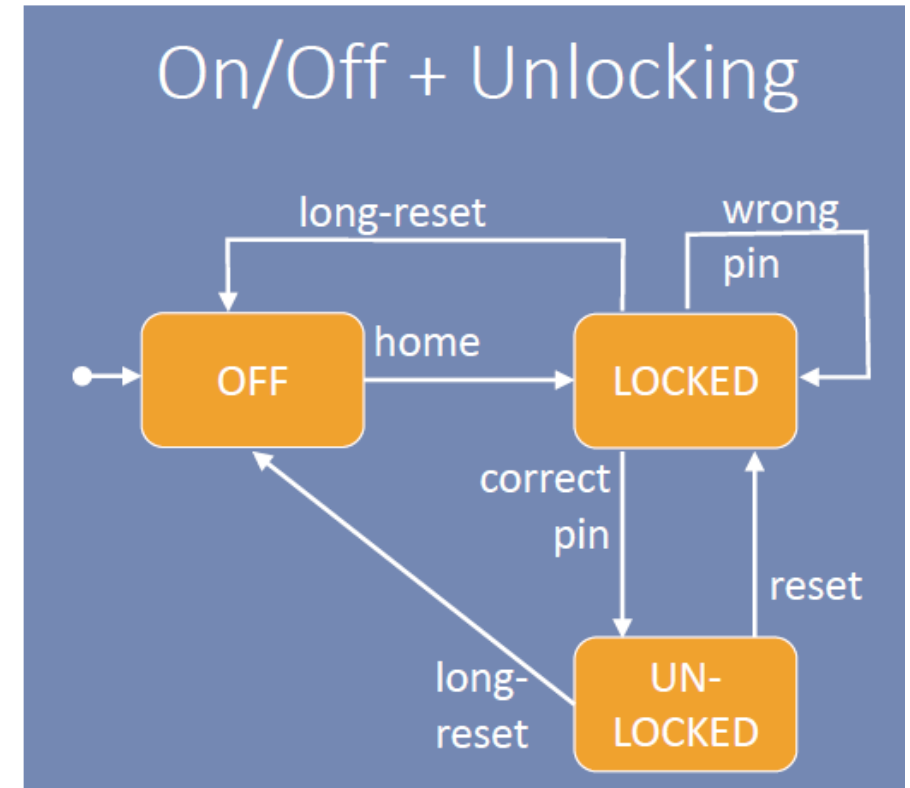
State Models: What is? and How to Draw?

- We can also model different behavior, for example unlocking the phone.
- Here we see a "locked" state, which we also mark as initial state, since by default a phone is locked
- We can unlock the phone by entering a correct pin, as shown in the diagram
- However, the entered pin code may also be wrong. In that case, the phone stays in the "locked" state.
- This is shown using a "self-transition", which goes from the LOCKED state back to the LOCKED state itself.
- Once we are unlocked, we can press the 'reset' button to lock the phone again to protect it against unintended use by others, modeled here as a transition from UNLOCKED to LOCKED.



State Models: What is? and How to Draw?

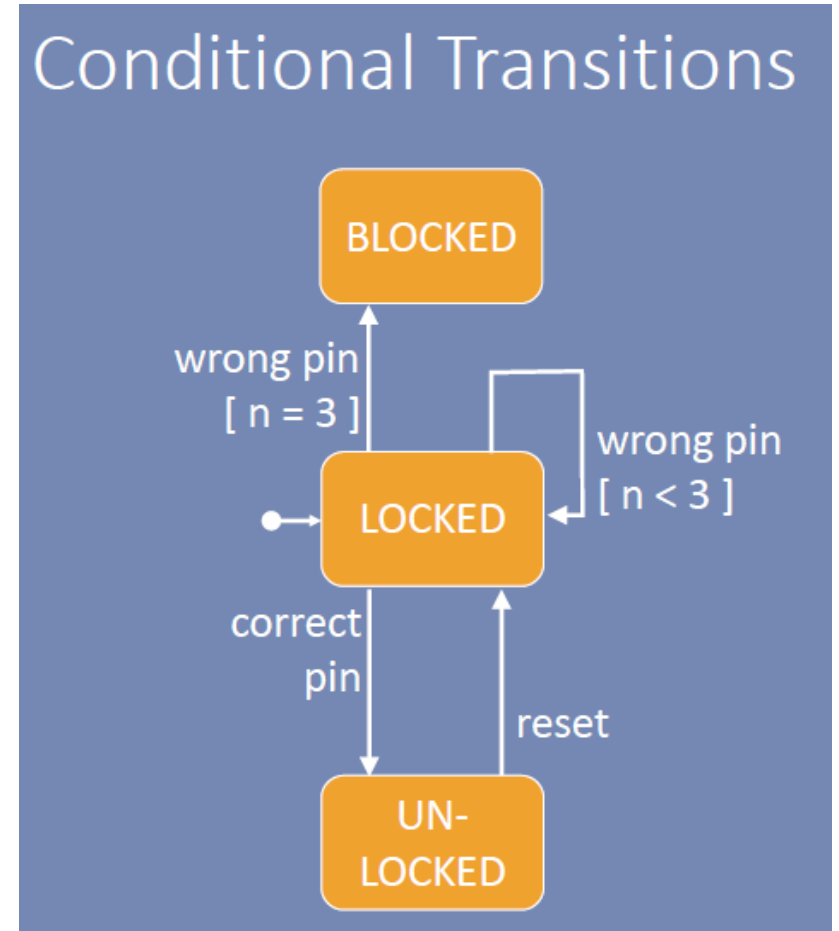
- Now we have modeled two aspects of the behavior of using your phone: switching it on, and unlocking it.
- We can combine these into a bigger diagram.
- Here we start with the unlocking state diagram. We can add the OFF state to this, together with the transition from OFF to the LOCKED state, which is the initial state of the unlocking diagram
- Pressing the reset button in any state switches off the phone. Therefore, for now we add Two transitions that go back to the OFF state: One starting in the LOCKED state at the top, and one in the UNLOCKED state at the bottom.
- The diagram has now become a little more complex, with three states, six transitions, and five different event types



State Models: What is? and How to Draw?

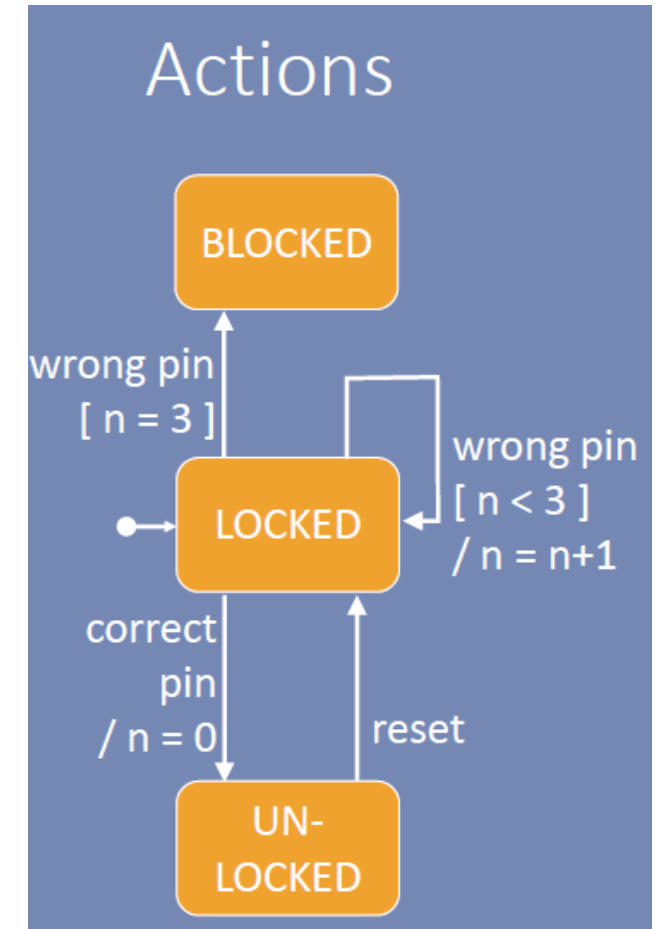
➤ Conditional Transition

- ❑ The condition, written within square brackets along with “wrong pin” event, states that the number of attempts (denoted by n) should be less than 3
- ❑ After entering a wrong pin three times, the phone gets blocked, as indicated here.
- ❑ Thus, the LOCKED state has two outgoing edges both triggered by the "wrong pin" event, and the condition determines what the resulting state is.



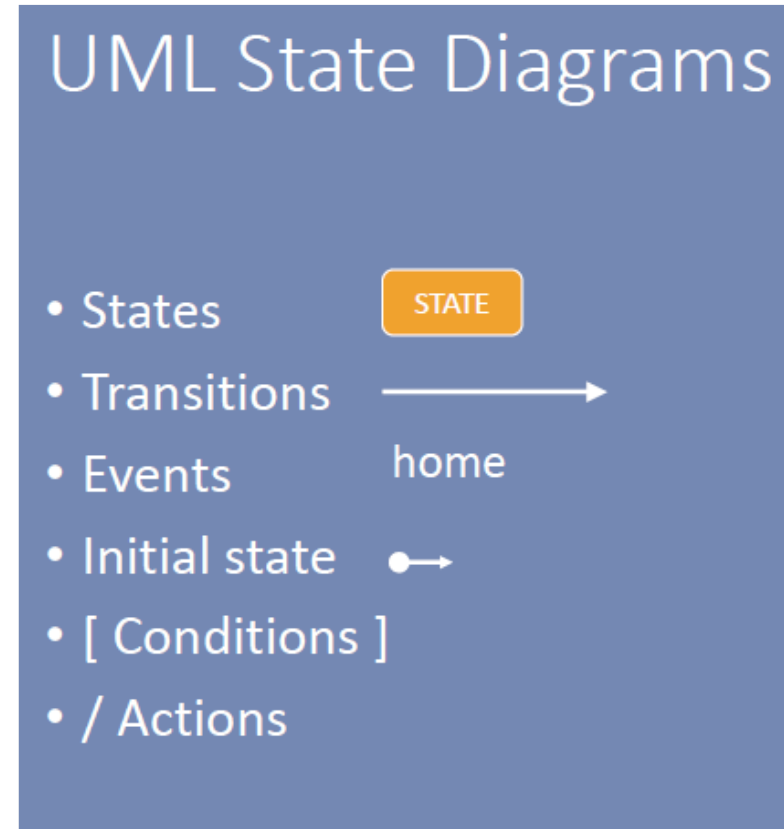
State Models: What is? and How to Draw?

- Transitions can have "actions". These are denoted by a forward slash "/", followed by the action
- In this example, we have added two actions, both manipulating the attempt counter "n".
 - ❑ If we enter a wrong pin, we increment the counter.
 - ❑ If we enter a correct pin, we reset it to zero.



State Models: basics of modeling behavior with UML state diagrams

- The notation that we are using is that of the Unified Modeling Language, UML.
- UML supports various diagram types, one of which is the state diagram that we are using.
- A state defines that the system is in a stable condition from which it will only change if it is stimulated by an event or same kind
- A transition is a change from one state to another.
 - ❑ The state change will be triggered by an event (e.g. pressing a button or switching a switch)
- The transition will be labelled with the event that caused it and any actions that arise from it
- Transitions are caused by events and they may generate outputs and/or changes of state



UML State Diagrams Notations

State Model

- An event is anything that acts as a trigger for a change
 - ❑ It could be an input to the system
 - ❑ It could be something inside the system, that changes for some reason
 - ❑ In some cases, events generate output
 - ❑ In some cases, events change the system's internal state without generating an output
 - ❑ In some cases, events can do both
 - ❑ Events sometimes associated with Conditions
- A model is an abstraction.
- The models we write have one particular use in mind.
- These models help us to derive test cases to verify the behavior of the system under test

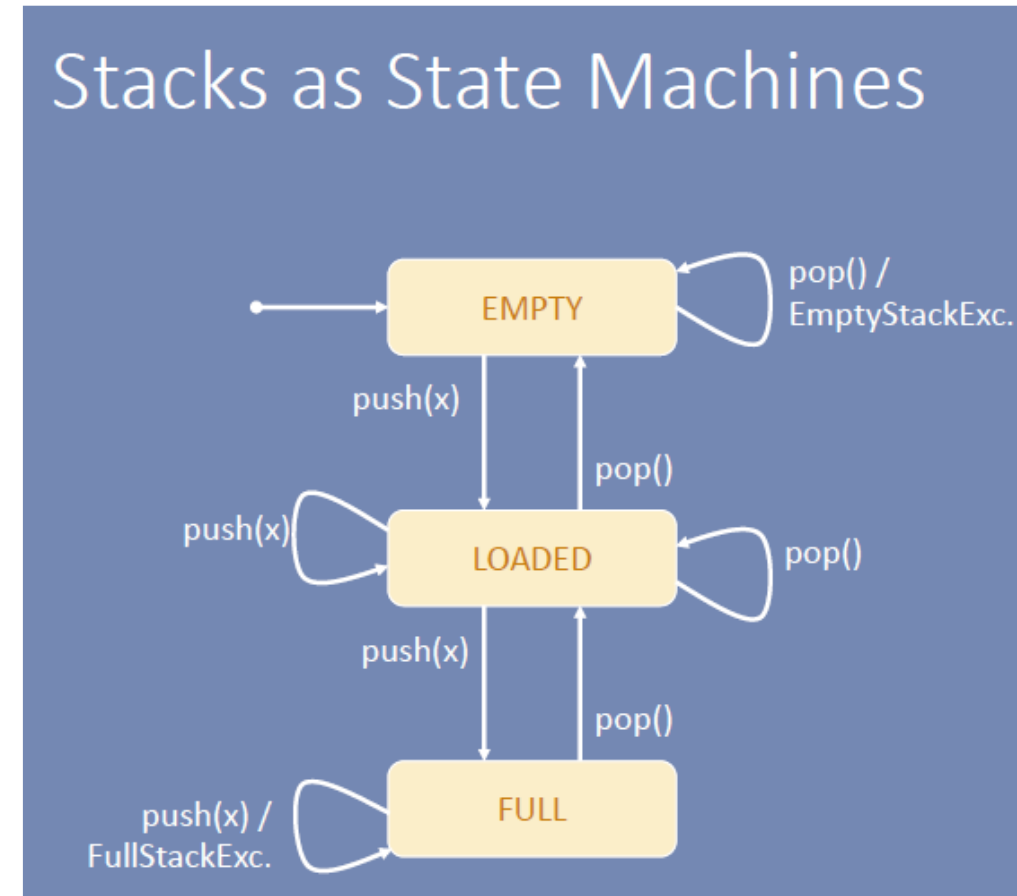
UML State Diagrams

- States 
- Transitions 
- Events 
- Initial state 
- [Conditions]
- / Actions

UML State Diagrams Notations

State Model for Stack: An Example

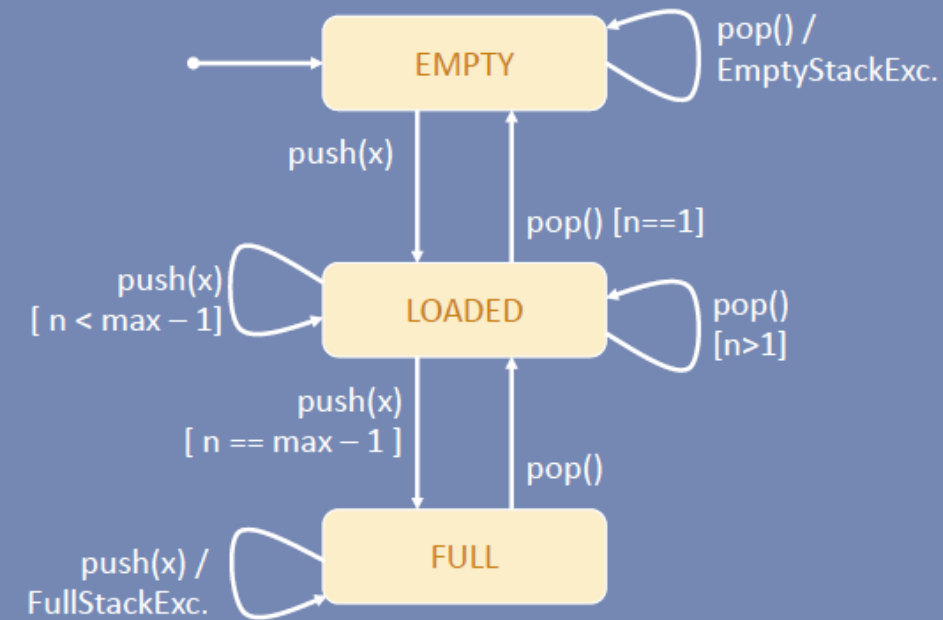
- Let's model a stack using a state machine.
- We start with a stack of nothing, in an Empty state.
- In the empty state, we can do one thing: namely add (push) an item, called x, on the stack.
- The stack then is loaded, and contains at least one element.
- In the loaded state, we can continue pushing elements, which will cause us to stay in the loaded state.
- Furthermore, we can pop elements off the stack, which for full stacks will keep us in the loaded state.
- If we pop the last element, the stack moves from the loaded back to the empty state.



State Model for Stack: An Example

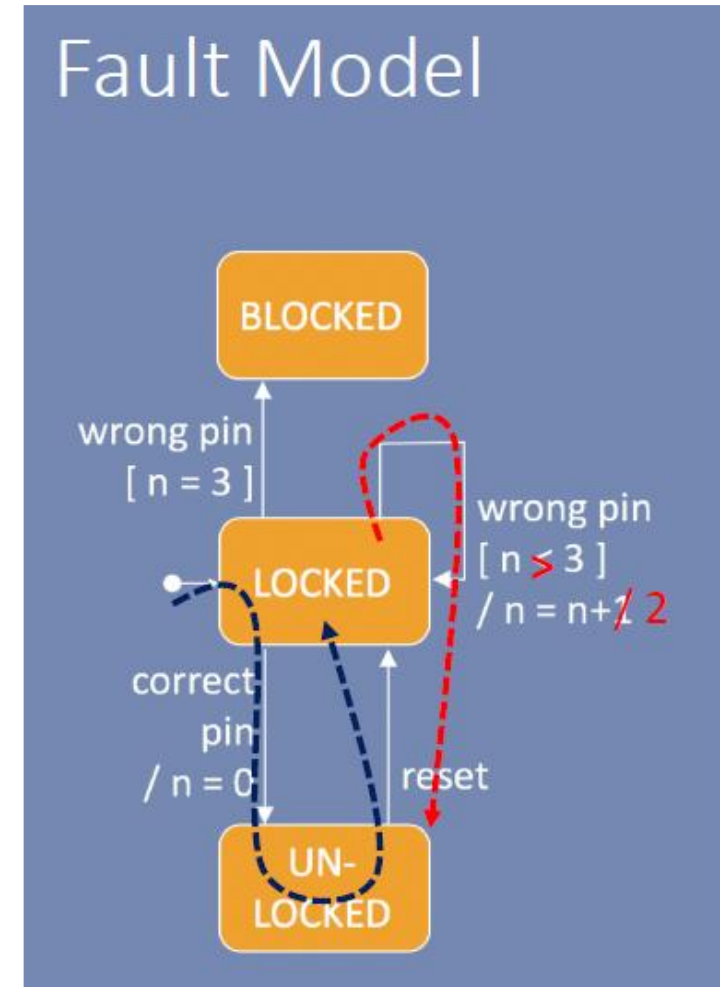
- We will assume that our stack is special and has a maximum size. In other words, the stack can be FULL after too many pushes.
- This is shown here through a condition labelled with the push event from LOADED to FULL.
- We can leave the full state by popping an element from it.
- The EMPTY state is special in the sense that popping from an empty state should not be possible.
- This modelled by adding an action to the pop action in the EMPTY state: the action is to raise a specific exception
- Likewise, the FULL state is special in the sense that no push is allowed. If there is such a push, our action is to raise the full stack exception.
- Some conditions have been added to disambiguate the pop and push actions

Stacks as State Machines



A fault Model

- Let's consider what might go wrong if our system should implement a faulty state machine.
- Here is our phone example again, with some faults drawn on top of it.
 - ❑ The red arrow goes from locked to unlocked, with a "wrong pin" event.
 - ❑ The general fault here is that a transition leads to the wrong state.
 - ❑ If there is a wrong pin event, the state should either be locked or blocked, but not unlocked.
 - ❑ Likewise, the condition may be incorrectly implemented ($>$ instead of $<$ condition), or the action may be wrong, incrementing by two instead of by just one.
 - ❑ A more subtle fault is illustrated by the black path, which goes through multiple states.
 - ❑ There is nothing wrong with this path, but it is somewhat tricky in the sense that the locked state is visited twice.



Test Coverage criteria for State Transition Testing

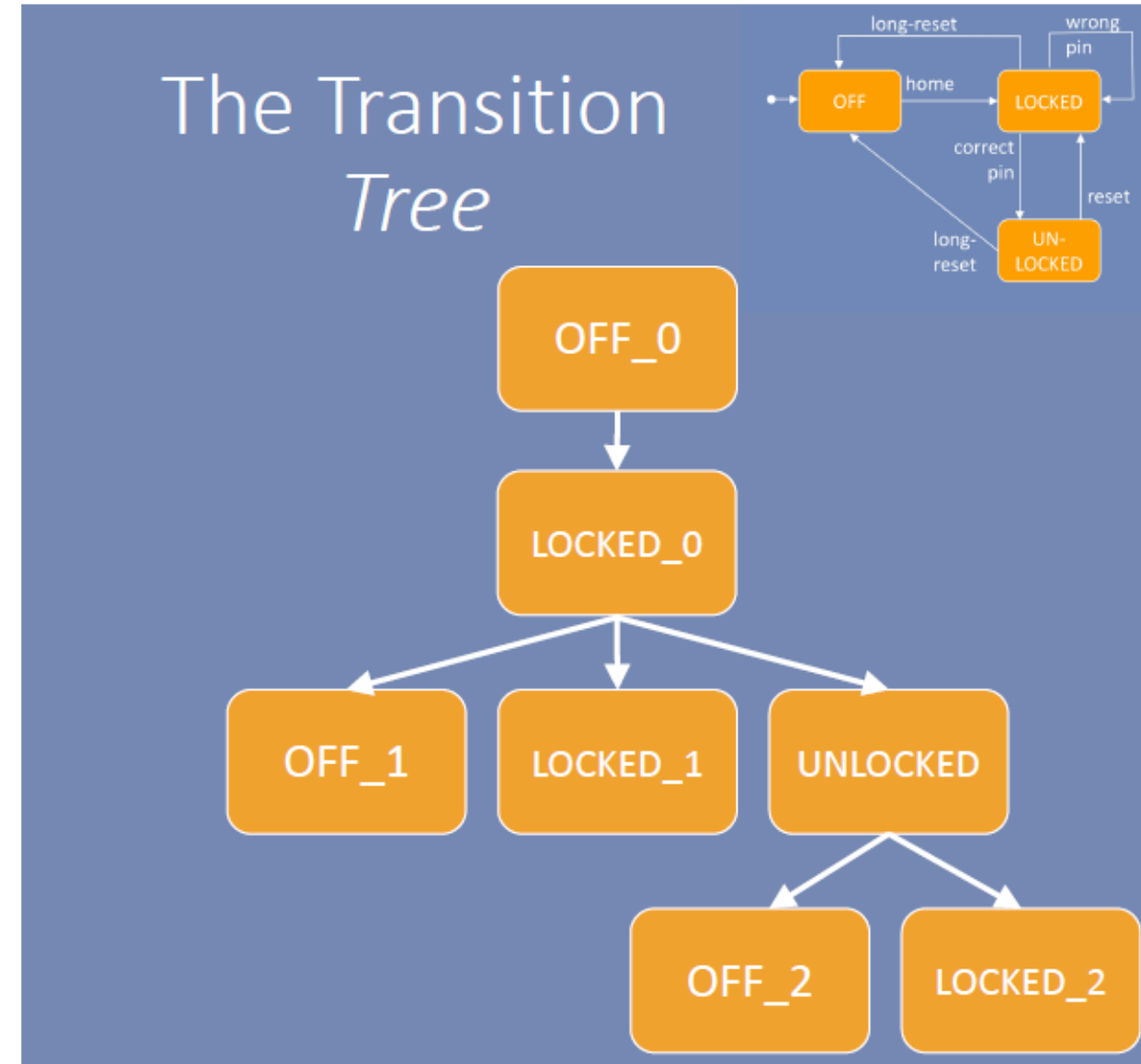
- To test for these types of faults, various forms of state machine specific test adequacy criteria can be used.
- Basic state coverage is the simplest, which just ensures that every state is reached once.
- Transition coverage is a bit stronger, which insists that every transition is covered.
- Path coverage insists that sequences of transitions , which are paths through the state machine, are tested .
- If state machines have loops, then each loop is exercised at least once in the case of Path Coverage.

State Machine Test Adequacy

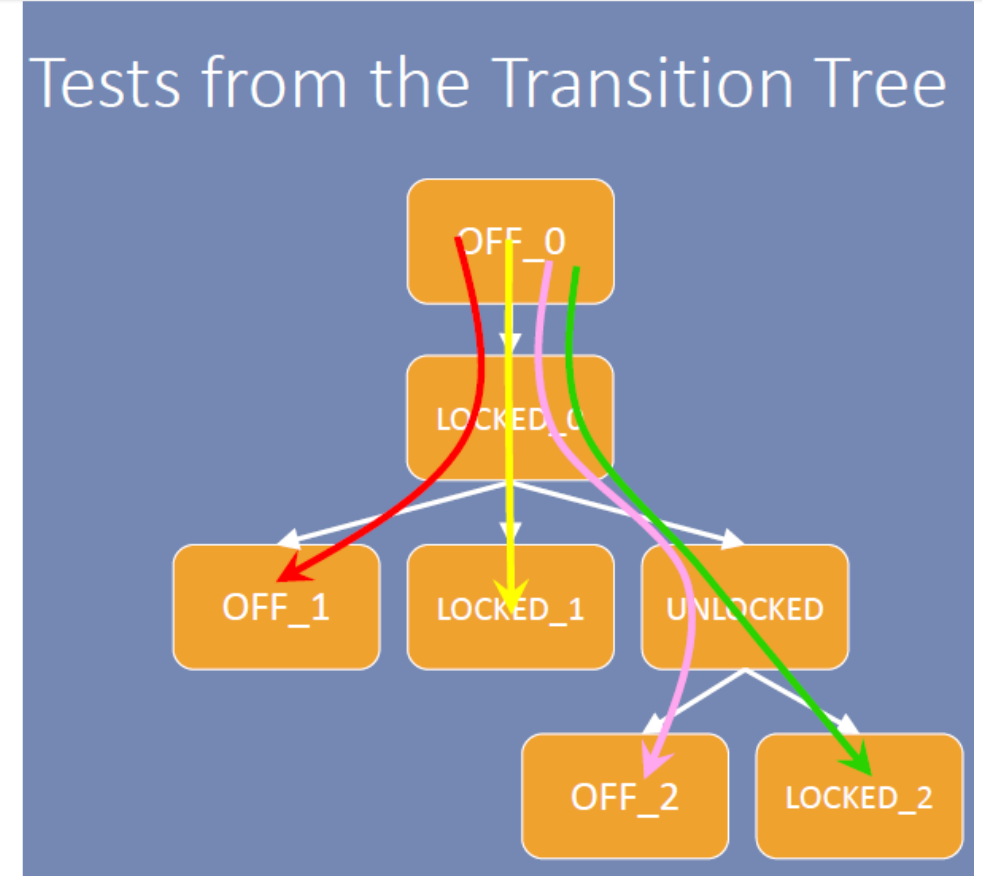
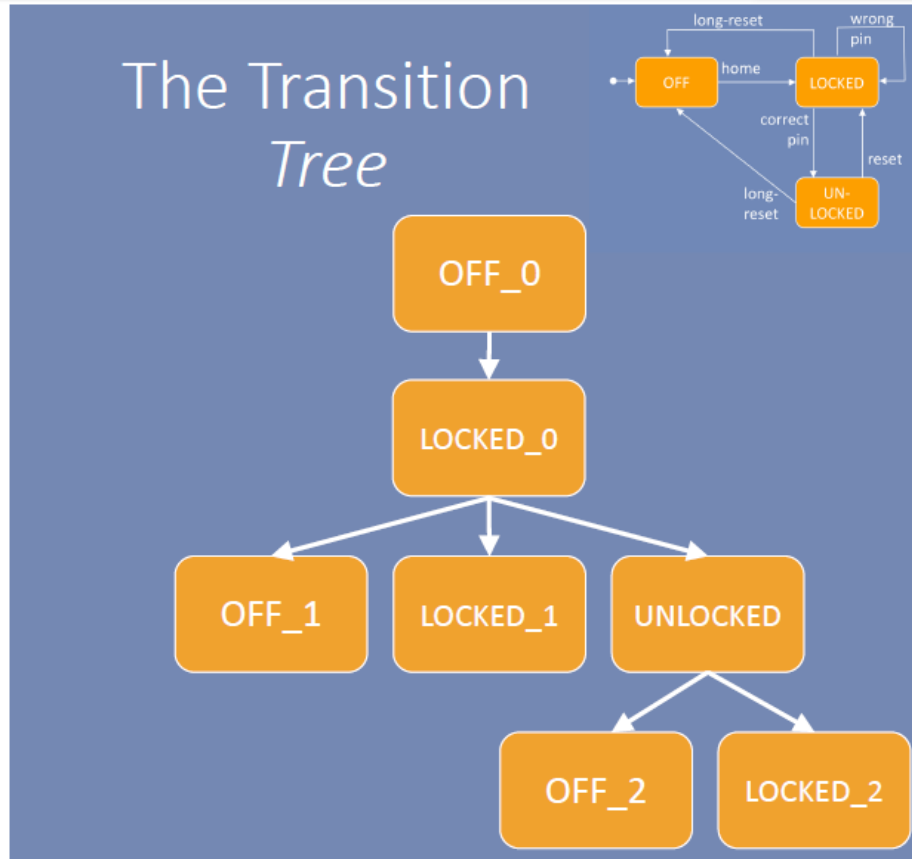
- State coverage:
 - Reach every state
- Transition coverage
 - Exercise every transition
- Path coverage
 - Exercise sequence of transitions

State Transition Tree

- A full test case will typically cover a sequence of transitions, i.e., it will exercise a path through the state machine.
- As there are potentially infinitely many paths we will need some approach to decide which paths to exercise.
- Our way to do that is by creating a transition tree, which spans the graph.
- The figure represents the transition tree for the example of mobile phone's off-on and lock-unlock behavior
- Once we have the tree, we can use it to derive test cases. Each test case then is a path from the root of the tree to one of the leaves.
- From the specified transition tree in the figure we get 4 test cases.



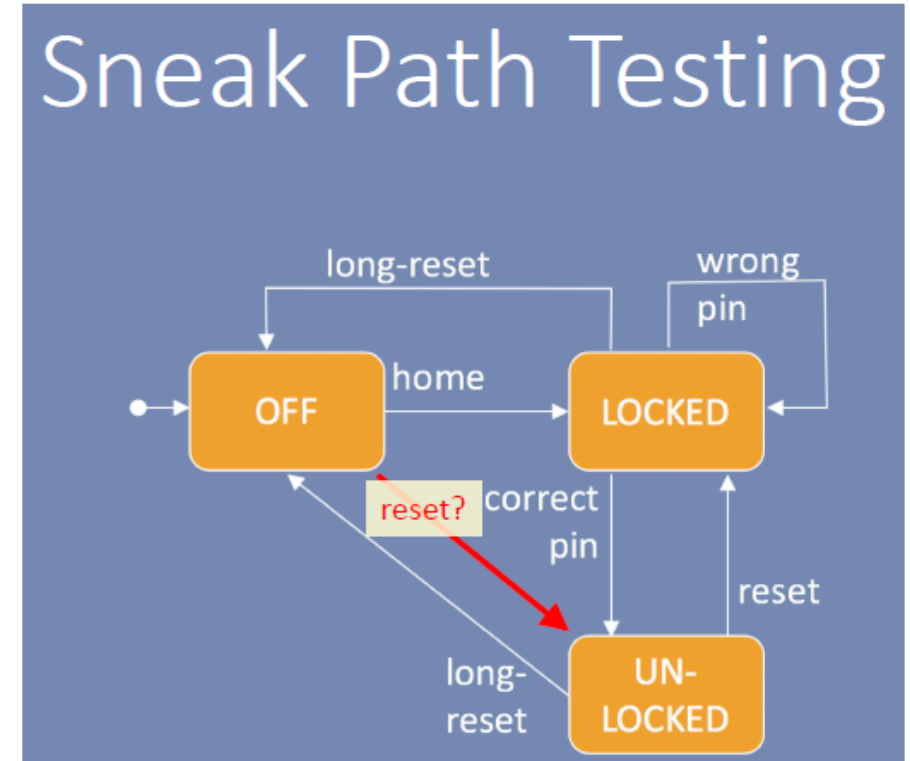
State Transition Tree



- We show the four test cases, with different colors.
- For example, the red path gives the simplest test case, corresponding to switching the phone on and off again, without unlocking it.
- The test path that also does the unlocking is the drawn in pink.

State Transition Tree

- Using transition tree, we can represent specified behavior in state models
- But what about unspecified behavior?
 - ❑ What if the system accidentally (or secretly) implements additional transitions?
 - ❑ For example, what if the system would allow going from OFF directly to unlocked, circumventing the pin code mechanism?
 - ❑ That would be a security problem, which we want to avoid.

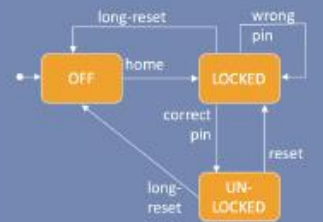


State Transition Table

- To check for such "sneak paths", a tabular representation of the state machine should be created.
 - ❑ That tabular form is known as State Transition Table
 - ❑ Sneak paths represent the null transition
 - ❑ The transition table specified in the figure is for simple phone example
 - ❑ Rows of a transition table represent States and Columns represent events
 - ❑ If a state has a transition with the given event, we indicate the corresponding cell with the outgoing state

Transition Table

STATES	events				
	home	long-reset	reset	wrong-pin	correct-pin
OFF	LOCKED				
LOCKED		OFF		LOCKED	UNLOCKED
UNLOCKED		OFF	LOCKED		

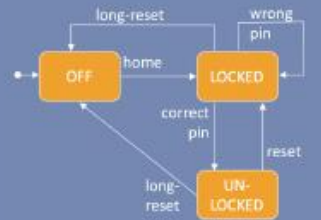


State Transition Table

- For example, in our state machine we can move from OFF to LOCKED by clicking the home button.
- This is indicated in the top left pink cell.
- No other events are possible from the OFF state which is why the remaining cells in the OFF row are empty.
- A table like this contains all the information from the diagram.
- Yet at the same time, the empty cells are directly visible. These empty cells correspond to "sneaky" transitions.

Transition Table

STATES	events				
	home	long-reset	reset	wrong-pin	correct-pin
OFF	LOCKED				
LOCKED		OFF		LOCKED	UNLOCKED
UNLOCKED		OFF	LOCKED		



Sneak Path Testing

- To test for such sneaky transitions we use State Transition table as follows.
- First we need to determine the intended behavior for empty cells.
- A common option is to simply ignore the event. An alternative possibility is to raise an exception.
- We start by bringing the system in a particular state, for example the LOCKED state.
- From there we trigger the two unspecified events (corresponding to home and reset in the empty cells).
- We then check that these events have no observable effect, and that the system stays in the LOCKED state.
- We repeat this for all empty cells: For our running example this then results in nine additional sneak path test cases.

Sneak Path Testing

- Create transition table
- Determine effects of empty cells
 - Default: do nothing

- Test each empty cell:
 - Default action
 - No state change

STATES	events				
	home	long-reset	reset	wrong-pin	correct-pin
OFF	LOCKED				
LOCKED		OFF		LOCKED	UNLOCKED
UNLOCKED		OFF	LOCKED		

- Verifies that illegal transitions cannot occur

Sneak Path Testing

- To bring the system in a particular state we can again make use of the transition tree.
- That tree tells us exactly which path to take from the top to reach a given state.
- The resulting "sneak path test suite" helps us to verify that illegal transactions cannot occur.

Sneak Path Testing

- Create transition table
- Determine effects of empty cells
 - Default: do nothing
- Test each empty cell:
 - Default action
 - No state change

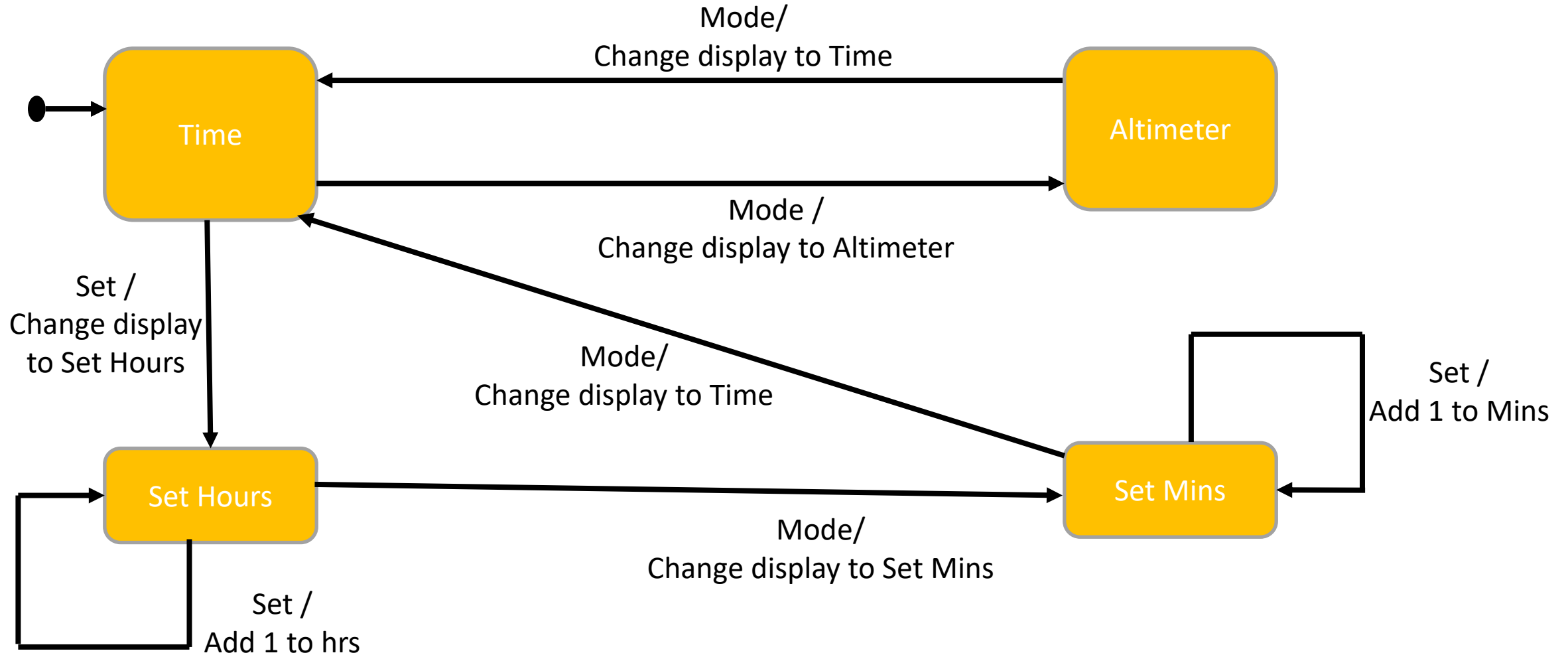
STATES	events				
	home	long-reset	reset	wrong-pin	correct-pin
OFF	LOCKED				
LOCKED		OFF		LOCKED	UNLOCKED
UNLOCKED		OFF	LOCKED		

- Verifies that illegal transitions cannot occur

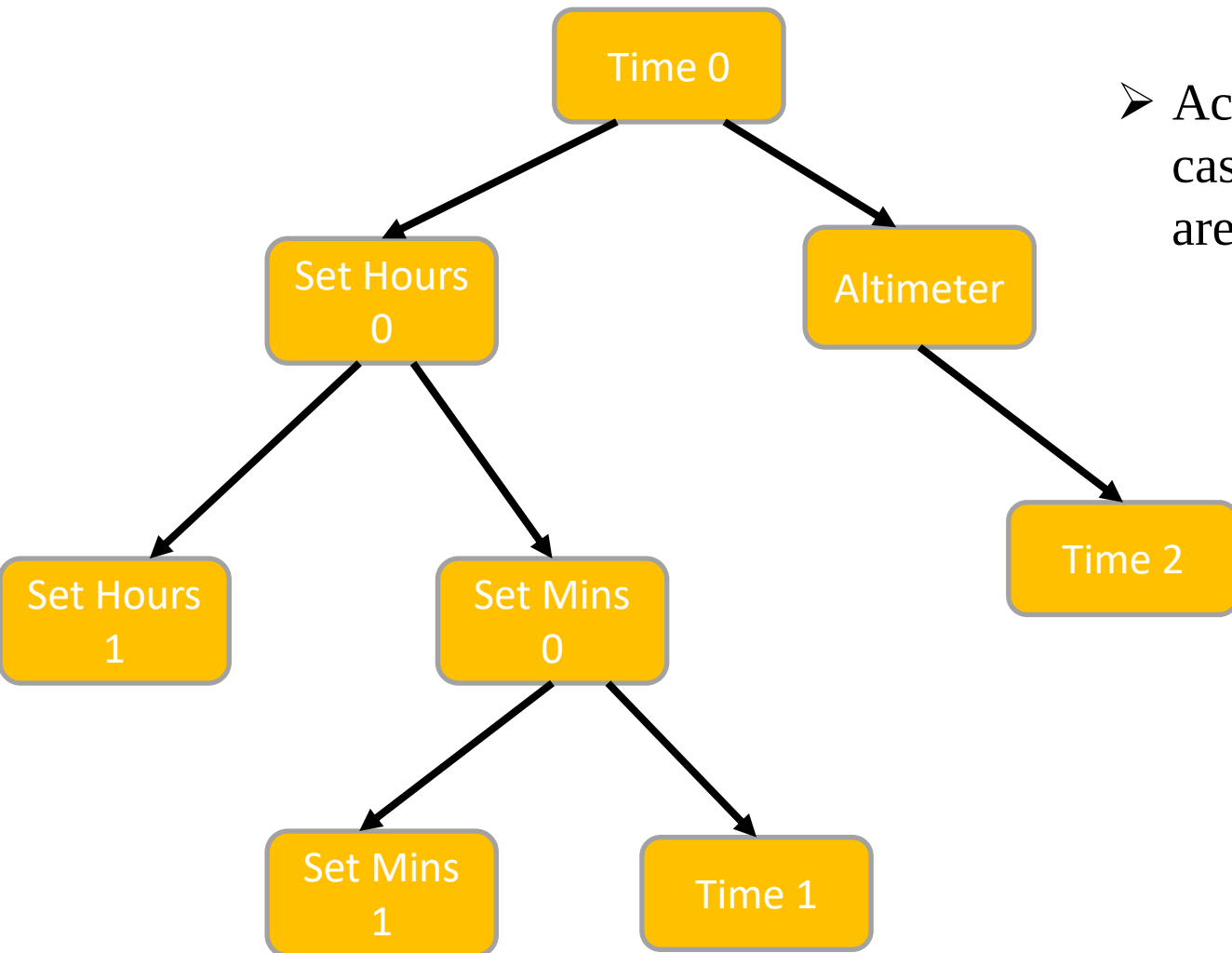
Example

- A hill-walker's watch has two modes
 - ☐ Time
 - ☐ Altimeter
- In Time mode,
 - ☐ pressing the Mode switch causes the watch to switch to Alt mode and display Current Altimeter.
 - ☐ pressing the Set button the watch is transited into Set Hrs and show the current hours
 - ❖ when is in SetHrs, the Hrs display can be incremented by pressing the Set button
 - ❖ when is in SetHrs, by pressing Mode switch, the watch transits to Set Mins mode and show the current minutes
 - When is in Set Mins mode, pressing the Set button, Mins display increment
 - When is in Set Mins mode, pressing the Mode button, transits back to Time mode and display current time.
- In Alt mode,
 - ☐ pressing Mode again returns to Time mode and display current time.
 - ☐ Pressing Set button has no effect

State Model for Example -1



State Transition Tree for Example -1



➤ According to the transition tree, this example has 4 test cases to test sequences of transition. Those sequences are

- ☐ Time 0 - Set Hours 0 - Set Hours 1
- ☐ Time 0 - Set Hours 0 - Set Mins 0 - Set Mins 1
- ☐ Time 0 - Set Hours 0 - Set Mins 0 – Time 1
- ☐ Time 0 – Altimeter - Time 2

State Transition Table for Example -1

	Mode	Set
Time	Altimeter/Change Display to Altimeter	Set Hrs/Change Display to Set Hrs
Altimeter	Time/Change display to time	
Set Hrs	Set Mins/Change display to Set Mins	Set Hrs/Add 1 to Hrs
Set Mins	Time/Change display to Time	Set Mins/Add 1 to Mins

- We can write both changed state and actions in table cells
- This example has only one sneak path test case related to the cell highlighted in red color.
- Therefore for example 1 there are total 5 test cases including sneak path test case
 - ❑ Time 0 - Set Hours 0 - Set Hours 1
 - ❑ Time 0 - Set Hours 0 - Set Mins 0 - Set Mins 1
 - ❑ Time 0 - Set Hours 0 - Set Mins 0 – Time 1
 - ❑ Time 0 – Altimeter - Time 2
 - ❑ Time 0-Altimeter-Null (**Sneak Path Test Case**)